

AULA 3

DTOs e Validação de Dados



OBJETIVOS DA AULA



AO FINAL DESSA AULA, VOCÊ VAI SER CAPAZ DE...

- Entender o que é um DTO e por que ele não é uma entidade de banco
- Usar decorators do class-validator pra declarar regras de validação
- Configurar o ValidationPipe globalmente na aplicação
- Entender o papel do class-transformer na conversão do payload



O QUE É UM DTO



DATA TRANSFER OBJECT

```
export class CreateUserDto {  
  name: string;  
  email: string;  
}
```

Sozinho, isso só existe em tempo de compilação — em runtime o JavaScript não sabe nada sobre esses tipos.

O QUE UM DTO NÃO É

- Não é uma entidade de banco de dados
- É um contrato de comunicação entre camadas/sistemas — define o formato do que entra ou sai da API
- Por convenção do Nest, é uma classe — não uma interface (interfaces somem em runtime, o Nest não consegue usá-las nos pipes)



DECORATORS DE VALIDAÇÃO



CLASS-VALIDATOR NA PRÁTICA

```
export class CreateUserDto {  
  @IsString()  
  @IsNotEmpty()  
  name: string;  
  
  @IsEmail()  
  email: string;  
}
```

```
npm install class-validator class-transformer
```

DECORATORS MAIS COMUNS

```
@IsString()  @IsInt()   @IsEmail()  
@IsOptional() @IsNotEmpty()  
@MinLength() / @MaxLength()  
@Min() / @Max()  @IsUUID()
```

Cada decorator declara uma regra. Combine quantos precisar na mesma propriedade.



VALIDATIONPIPE



CONFIGURAÇÃO GLOBAL (main.ts)

```
app.useGlobalPipes(  
  new ValidationPipe({  
    whitelist: true,  
    forbidNonWhitelisted: true,  
    transform: true,  
  } ),  
);
```

O QUE CADA OPÇÃO FAZ

- **whitelist** — remove do payload qualquer campo que não esteja no DTO
- **forbidNonWhitelisted** — rejeita com 400 se vier campo extra, em vez de ignorar
- **transform** — converte o payload plano numa instância de verdade do DTO, com os tipos certos
- Se a validação falhar, o Nest já devolve 400 com a lista de erros, sem código extra



CLASS-TRANSFORMER



DE OBJETO PLANO PRA INSTÂNCIA TIPADA

```
plainToInstance(CreateUserDto, { name: 'Ana', email: 'ana@x.com' })  
// vira uma instância real de CreateUserDto — só assim o  
// class-validator consegue enxergar os decorators e validar
```

POR QUE ISSO É NECESSÁRIO

- Todo payload que chega pela rede é um objeto JavaScript puro, sem tipo — o ValidationPipe usa o class-transformer por baixo dos panos pra resolver isso antes de validar
- O mesmo pacote também ajuda a controlar o que sai na resposta (ex: nunca devolver senha) — assunto que volta na Aula 9

